

Depth_grid_processing

April 12, 2026

```
[1]: from pathlib import Path
import numpy as np
import pandas as pd
import rasterio
import matplotlib.pyplot as plt
```

```
[5]: import pandas as pd
from pathlib import Path

file_path = Path("raw_data/north_slope_depth_grids/NSbasement.XYZ")

df = pd.read_fwf(
    file_path,
    skiprows=8,
    header=None
)

print(df.head())
print(df.shape)
```

	0	1	2	3	4
0	-630000	2000000	-611	-168.67649	67.32879
1	-627000	2000000	-791	-168.60884	67.33470
2	-624000	2000000	-1092	-168.54116	67.34059
3	-621000	2000000	-1391	-168.47344	67.34646
4	-618000	2000000	-1668	-168.40570	67.35229

(90480, 5)

```
[6]: df.columns = ["x", "y", "z", "lon", "lat"]
```

```
[7]: grid = df.pivot(index="y", columns="x", values="z")

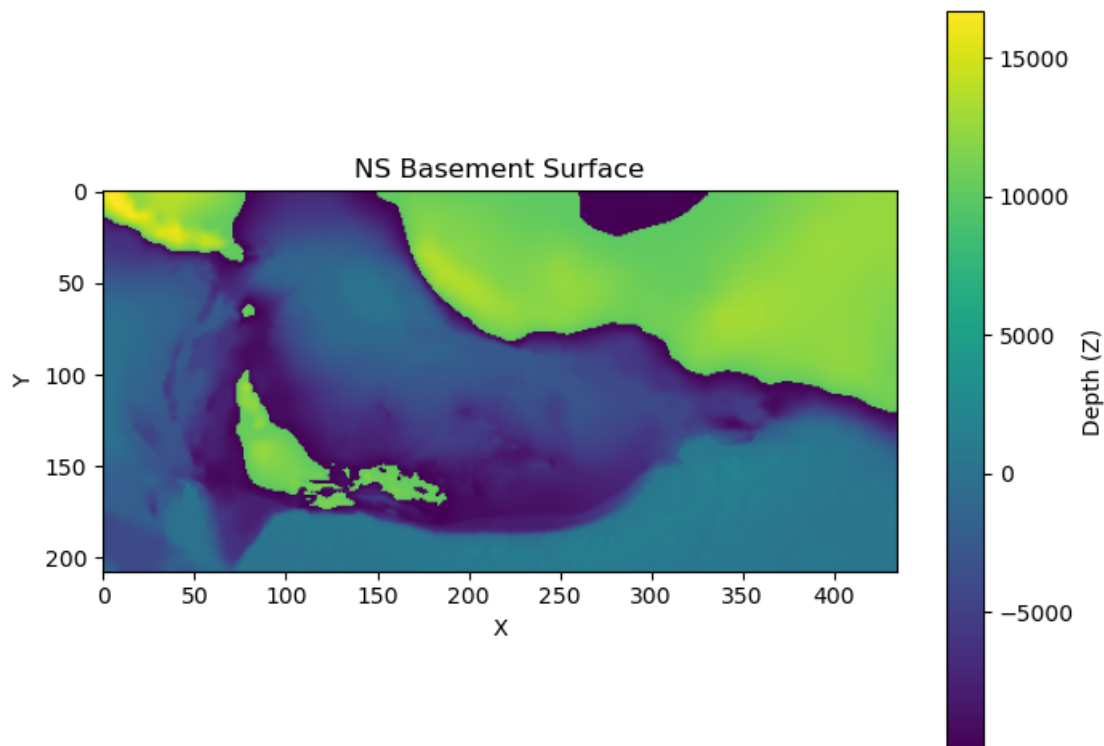
# flip so north is up visually
grid = grid.sort_index(ascending=False)

print(grid.shape)
```

(208, 435)

```
[8]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.imshow(grid, cmap="viridis")
plt.colorbar(label="Depth (Z)")
plt.title("NS Basement Surface")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



```
[9]: def load_xyz_grid(file_path):
    rows = []

    with open(file_path, "r") as f:
        for line in f.readlines()[8:]:
            parts = line.strip().split()
            if len(parts) == 5:
                rows.append(parts)

    df = pd.DataFrame(rows, columns=["x", "y", "z", "lon", "lat"])
    df = df.apply(pd.to_numeric, errors="coerce").dropna()
```

```

grid = df.pivot(index="y", columns="x", values="z")
grid = grid.sort_index(ascending=False)

return grid

```

```

[10]: base_dir = Path("raw_data/north_slope_depth_grids")

grids = {}

for file in base_dir.glob("*.XYZ"):
    print("Loading:", file.name)
    grids[file.stem] = load_xyz_grid(file)

print(grids.keys())

```

```

Loading: NStopo-basement.XYZ
Loading: NStopo.XYZ
Loading: NSbasement.XYZ
Loading: NSLCU.XYZ
Loading: NSshublik-basement.XYZ
Loading: NStopo-LCU.XYZ
Loading: NSshublik.XYZ
Loading: NSLCU-Shublik.XYZ
dict_keys(['NStopo-basement', 'NStopo', 'NSbasement', 'NSLCU', 'NSshublik-
basement', 'NStopo-LCU', 'NSshublik', 'NSLCU-Shublik'])

```

```

[11]: for name, g in grids.items():
        print(name, g.shape)

```

```

NStopo-basement (208, 435)
NStopo (208, 435)
NSbasement (208, 435)
NSLCU (208, 435)
NSshublik-basement (208, 435)
NStopo-LCU (208, 435)
NSshublik (208, 435)
NSLCU-Shublik (208, 435)

```

```

[12]: import numpy as np

master = grids["NSbasement"]

x_vals = master.columns.to_numpy()
y_vals = master.index.to_numpy()

X, Y = np.meshgrid(x_vals, y_vals)

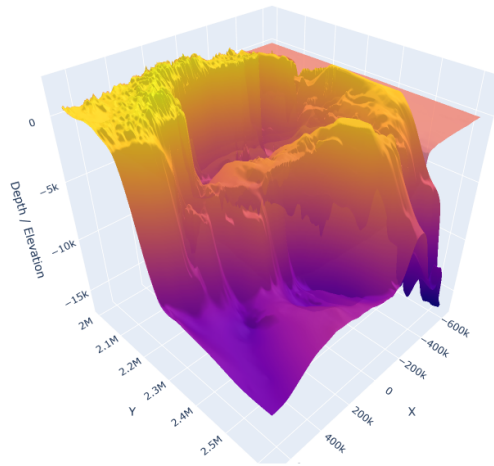
print(X.shape, Y.shape, master.shape)

```

(208, 435) (208, 435) (208, 435)

```
[13]: def grid_to_surface_arrays(grid):  
      z = grid.to_numpy()  
      x = grid.columns.to_numpy()  
      y = grid.index.to_numpy()  
      X, Y = np.meshgrid(x, y)  
      return X, Y, z
```

```
[15]: import plotly.graph_objects as go  
  
fig = go.Figure()  
  
surface_names = ["NStopo", "NSLCU", "NSshublik", "NSbasement"]  
  
for name in surface_names:  
    X, Y, Z = grid_to_surface_arrays(grids[name])  
  
    fig.add_trace(go.Surface(  
        x=X,  
        y=Y,  
        z=Z,  
        name=name,  
        showscale=False,  
        opacity=0.75,  
        hovertemplate=(  
            f"<b>{name}</b><br>"  
            "X: {x}<br>"  
            "Y: {y}<br>"  
            "Z: {z}<extra></extra>"  
        )  
    ))  
  
fig.update_layout(  
    title="North Slope Structural Surfaces",  
    scene=dict(  
        xaxis_title="X",  
        yaxis_title="Y",  
        zaxis_title="Depth / Elevation",  
    ),  
    height=800,  
    width=1100  
)  
  
fig.show()
```



```
[16]: surface_settings = {
    "NSstopo": {"opacity": 0.35, "colorscale": "Greens"},
    "NSLCU": {"opacity": 0.55, "colorscale": "Blues"},
    "NSshublik": {"opacity": 0.65, "colorscale": "Cividis"},
    "NSbasement": {"opacity": 0.80, "colorscale": "Viridis"},
}

fig = go.Figure()

for name, settings in surface_settings.items():
    X, Y, Z = grid_to_surface_arrays(grids[name])

    fig.add_trace(go.Surface(
        x=X,
        y=Y,
        z=Z,
        name=name,
        opacity=settings["opacity"],
        colorscale=settings["colorscale"],
        showscale=False,
        hovertemplate=(
            f"<b>{name}</b><br>"
            "X: %{x}<br>"
            "Y: %{y}<br>"
            "Z: %{z}<extra></extra>"
        )
    ))
```

```

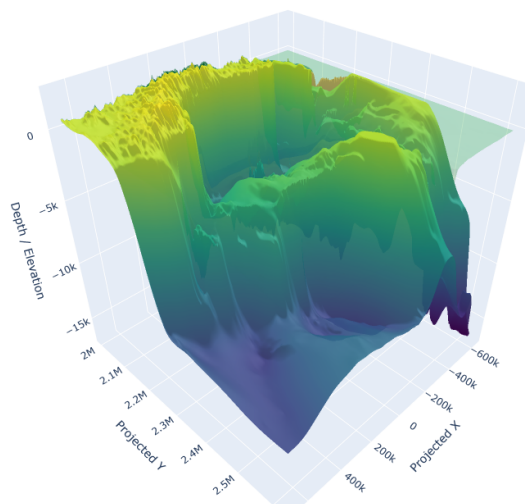
))

fig.update_layout(
    title="North Slope Structural Framework",
    scene=dict(
        xaxis_title="Projected X",
        yaxis_title="Projected Y",
        zaxis_title="Depth / Elevation"
    ),
    height=850,
    width=1200
)

fig.show()

```

North Slope Structural Framework



```

[17]: def grid_to_long_df(grid, surface_name):
        df_long = (
            grid
            .stack()
            .reset_index()
        )
        df_long.columns = ["y", "x", "z"]
        df_long["surface"] = surface_name
        return df_long[["surface", "x", "y", "z"]]

```

```

surface_tables = []

for name in ["NStopo", "NSLCU", "NSshublik", "NSbasement"]:
    surface_tables.append(grid_to_long_df(grids[name], name))

surfaces_df = pd.concat(surface_tables, ignore_index=True)
surfaces_df.head()

```

```

[17]:
  surface      x      y      z
0  NStopo -630000 2621000 -67
1  NStopo -627000 2621000 -75
2  NStopo -624000 2621000 -82
3  NStopo -621000 2621000 -81
4  NStopo -618000 2621000 -74

```

```

[18]: surfaces_df.to_parquet("north_slope_structural_surfaces.parquet", index=False)

```

```

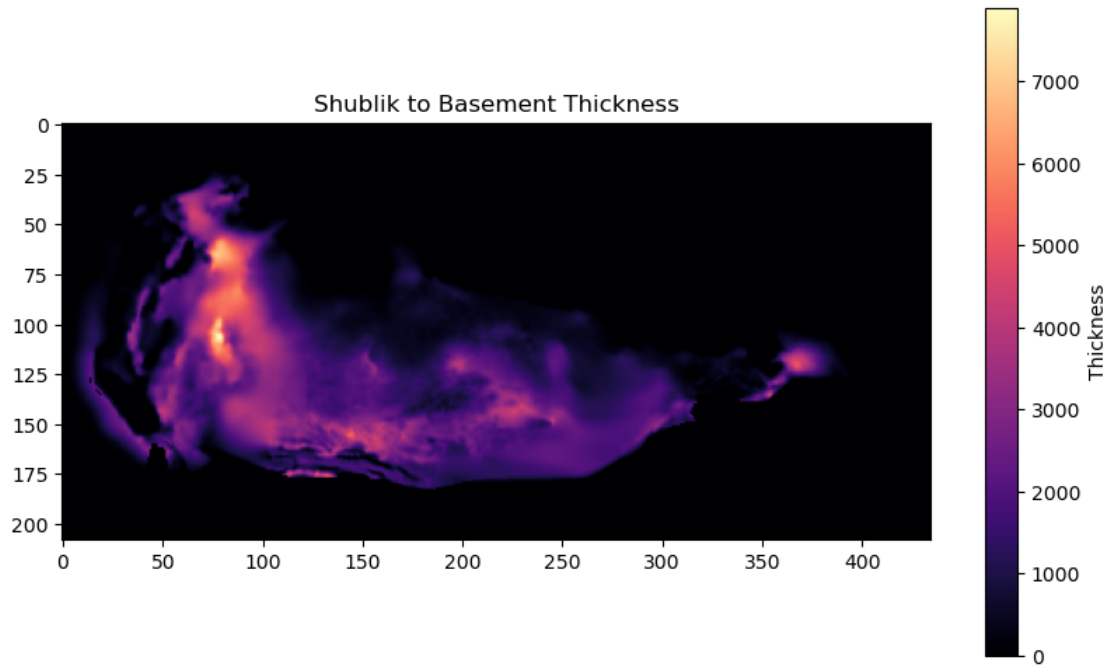
[19]: thickness_maps = {
    "Topo_minus_Basement": grids["NStopo"] - grids["NSbasement"],
    "Topo_minus_LCU": grids["NStopo"] - grids["NSLCU"],
    "LCU_minus_Shublik": grids["NSLCU"] - grids["NSshublik"],
    "Shublik_minus_Basement": grids["NSshublik"] - grids["NSbasement"],
}

```

```

[20]: plt.figure(figsize=(10, 6))
plt.imshow(thickness_maps["Shublik_minus_Basement"], cmap="magma")
plt.colorbar(label="Thickness")
plt.title("Shublik to Basement Thickness")
plt.show()

```



```
[22]: from pyproj import Transformer

# sample row from your dataframe
sample = df.iloc[0]
print(sample[["x", "y", "lon", "lat"]])

transformer = Transformer.from_crs("EPSG:4326", "EPSG:3338", always_xy=True)

x_test, y_test = transformer.transform(sample["lon"], sample["lat"])

print("XYZ file x,y: ", sample["x"], sample["y"])
print("Transformed x,y:", x_test, y_test)
print("Difference:      ", sample["x"] - x_test, sample["y"] - y_test)

x      -6.300000e+05
y       2.000000e+06
lon    -1.686765e+02
lat     6.732879e+01
Name: 0, dtype: float64
XYZ file x,y: -630000.0 2000000.0
Transformed x,y: -629973.259400802 1999963.7542702495
Difference:      -26.740599198034033 36.24572975048795
```

```
[ ]:
```